

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH



BÀI TẬP LỚN MÔN XÁC SUẤT THỐNG KÊ

Đề tài Xử lý dữ liệu

Phân Tích và Dự Đoán Giá Phát Hành GPU

Dựa Trên Các Thông Số Kỹ Thuật

GVHD: Huỳnh Thái Duy Phương

SV: Trần Gia Tường - 2313835

Trần Nhật - 2412490

Trương Minh Triết - 2413604

Vũ Đại Dương - 2410638

TP. HỒ CHÍ MINH, 2026

DANH SÁCH THÀNH VIÊN

STT	Họ và tên	MSSV	Mức độ đóng góp
1	Trần Gia Tường	2313835	100%
2	Trần Nhật	2412490	100%
3	Trương Minh Triết	2413604	100%
4	Vũ Đại Dương	2410638	100%



Mục lục

1	Đối tượng, Mục tiêu và Kết quả Hướng tới	4
1.1	Bộ dữ liệu được chọn	4
1.2	Bối cảnh thực tế	4
1.3	Mục tiêu cụ thể	5
1.4	Kết quả kỳ vọng	5
2	Cơ sở Lý thuyết	6
2.1	Giới thiệu bộ dữ liệu	6
2.2	Lựa chọn biến phân tích và giải thích	6
2.2.1	Quy trình lọc biến từ 34 thuộc tính ban đầu	7
2.2.2	Kiểm chứng quan hệ tương quan giữa biến số và giá GPU	10
2.2.3	Bảng tổng hợp 8 biến đã chọn	11
2.3	Các bước xử lý dữ liệu	12
2.3.1	Bước 1: Tải và khảo sát sơ bộ dữ liệu	12
2.3.2	Bước 2: Chọn lọc cột và ép kiểu dữ liệu	13
2.3.3	Bước 3: Phát hiện và xử lý ngoại lệ (Outliers)	13
2.3.4	Bước 4: Xử lý giá trị khuyết (Missing Values)	14
2.3.5	Bước 5 & 6: Mã hóa biến phân loại và Chuẩn hóa dữ liệu	16
2.4	Xây dựng mô hình thống kê	16
2.4.1	Mô hình hồi quy tuyến tính bội (Multiple Linear Regression)	16
2.4.2	Phân tích phương sai (Welch's ANOVA)	18
2.5	Sơ đồ tổng quan quy trình xử lý	18
3	Code và Xử lý Cụ thể	19
3.1	Bước 1: Tải và khảo sát sơ bộ dữ liệu	19
3.2	Bước 2: Chọn lọc cột và ép kiểu dữ liệu	20
3.3	Bước 3: Xử lý ngoại lệ	21



3.4	Bước 4: Xử lý giá trị khuyết	22
3.5	Bước 5: Mã hóa biến phân loại	23
3.6	Bước 6: Chuẩn hóa dữ liệu (z-score normalization)	23
3.7	Bước 7: Xây dựng mô hình thống kê	24
3.7.1	7.1. Mô hình hồi quy tuyến tính bội	24
3.7.2	7.2. Phân tích phương sai	26
4	Kết quả Thu được	27
4.1	Bước 1 — Tải và khảo sát sơ bộ dữ liệu	27
4.2	Bước 2 — Chọn lọc cột và ép kiểu dữ liệu	27
4.3	Bước 3 — Phát hiện và đánh giá ngoại lệ	27
4.4	Bước 4 — Xử lý giá trị khuyết	28
4.5	Bước 5 & 6 — Mã hóa biến phân loại và chuẩn hóa Z-score .	28
4.6	Thống kê mô tả biến phụ thuộc release_price	28
4.7	Ma trận tương quan Pearson	29
4.8	Kết quả ANOVA — So sánh giá giữa NVIDIA và AMD . . .	29
4.9	Kết quả mô hình hồi quy	29
4.9.1	Mô hình OLS cơ sở	29
4.9.2	Mô hình Log-Linear (Mô hình tinh chỉnh)	30
4.9.3	Diễn giải hệ số và kiểm định giả định	30
5	Kết luận và Đề nghị	31
5.1	Đánh giá mục tiêu nghiên cứu	31
5.1.1	Câu hỏi nghiên cứu đã được giải quyết	31
5.2	Hạn chế và đề xuất cải tiến	31
5.3	Ý nghĩa thực tiễn	32

1 Đối tượng, Mục tiêu và Kết quả Hướng tới

1.1 Bộ dữ liệu được chọn

Nhóm lựa chọn bộ dữ liệu **All_GPUs.csv**, được trích xuất từ tập dữ liệu “*Computer Parts (CPUs and GPUs)*” do tác giả Ilias Kkaf công bố trên nền tảng Kaggle. Đây là bộ dữ liệu thực tế, bao gồm thông số kỹ thuật chi tiết của hơn 3.400 card đồ họa (GPU) từ các nhà sản xuất lớn như NVIDIA, AMD và Intel, trải dài qua nhiều thế hệ sản phẩm.

Bộ dữ liệu thô chứa 34 thuộc tính mô tả đặc điểm phần cứng của GPU. Sau quy trình sàng lọc ba tầng dựa trên (i) tỷ lệ dữ liệu khuyết, (ii) trùng lặp/dẫn xuất giữa các cột, và (iii) liên quan tới cấu trúc định giá — chi tiết tại Mục 2.2.1 — nhóm chốt 8 biến đưa vào mô hình: **release_price** (biến phụ thuộc) cùng 7 biến độc lập gồm **tdp**, **memory_size**, **memory_bus**, **core_speed**, **memory_type**, **manufacturer**, **release_year**.

1.2 Bối cảnh thực tế

Thị trường GPU trong thập kỷ qua đã trải qua những biến động giá cực kỳ mạnh mẽ. Không chỉ là linh kiện phục vụ gaming, GPU ngày nay còn đóng vai trò trụ cột trong các lĩnh vực trí tuệ nhân tạo, khai thác tiền điện tử và điện toán hiệu năng cao. Sự cạnh tranh giữa hai ông lớn NVIDIA và AMD tạo ra một thị trường phân tầng rõ rệt: từ các sản phẩm tầm trung phổ thông cho đến những flagship đắt tiền hàng nghìn đô la như dòng TITAN hay Quadro của NVIDIA.

Trong bối cảnh đó, người mua GPU — dù là cá nhân hay doanh nghiệp — thường đối mặt với câu hỏi: “*Thông số kỹ thuật nào thực sự quyết định giá của một chiếc GPU? Và liệu cùng cấu hình đó, GPU của NVIDIA có đắt hơn AMD đáng kể không?*” Các câu hỏi này không chỉ mang tính học thuật mà còn có giá trị thực tiễn trực tiếp cho quyết định mua sắm và hoạch định ngân sách.

1.3 Mục tiêu cụ thể

Dự án đặt ra hai mục tiêu chính được triển khai song song:

Mục tiêu 1 — Xây dựng mô hình hồi quy dự đoán giá: Xây dựng mô hình hồi quy tuyến tính bội (Multiple Linear Regression) để định lượng tác động của từng thông số kỹ thuật lên giá phát hành GPU. Phương trình mô hình tổng quát có dạng:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_k X_k + \varepsilon$$

trong đó Y là giá phát hành (release_price), các X_i là các thông số kỹ thuật và ε là sai số ngẫu nhiên. Mô hình sẽ được tính chỉnh (bao gồm biến đổi logarit nếu cần) để đảm bảo các giả định thống kê cơ bản được thỏa mãn và tối ưu hóa khả năng giải thích (R^2).

Mục tiêu 2 — Phân tích ANOVA so sánh giá giữa các hãng: Áp dụng phân tích phương sai một chiều (ANOVA) để kiểm định xem có sự khác biệt có ý nghĩa thống kê về giá trung bình giữa GPU của NVIDIA và AMD hay không. Kiểm định hậu kiểm (Post hoc) sẽ được dùng để lượng hóa cụ thể mức chênh lệch giữa hai hãng.

1.4 Kết quả kỳ vọng

Sau khi hoàn thành phân tích, nhóm kỳ vọng đạt được các kết quả sau:

- **Mô hình hồi quy có ý nghĩa thực tiễn:** Phương trình dạng $\log(\hat{Y}) = \beta_0 + \beta_1 X_1 + \dots + \beta_k X_k$ giúp dự đoán giá GPU với độ chính xác cao ($R^2 \geq 70\%$), trong đó các hệ số β_i cho phép so sánh tầm quan trọng tương đối của từng thông số kỹ thuật đối với giá thành sản phẩm.
- **Xác định yếu tố định giá chủ đạo:** Kỳ vọng công suất tiêu thụ (TDP), dung lượng bộ nhớ và hãng sản xuất là các biến có tác động mạnh nhất đến giá, phản ánh đúng quy luật kinh tế học phần cứng.

- **Bảng chứng thống kê về chênh lệch giá NVIDIA – AMD:** Kiểm định ANOVA xác nhận NVIDIA có mức giá phát hành trung bình cao hơn AMD một cách có ý nghĩa thống kê. Kết quả định lượng mức chênh lệch này (bằng USD) sẽ là thông tin hữu ích cho người tiêu dùng khi đưa ra quyết định mua sắm.
- **Gợi ý thực tiễn cho người mua GPU:** Từ kết quả mô hình, người mua có thể đánh giá xem mức giá của một GPU có hợp lý so với các thông số kỹ thuật của nó hay không, từ đó lựa chọn được sản phẩm tối ưu trong ngân sách của mình.

2 Cơ sở Lý thuyết

2.1 Giới thiệu bộ dữ liệu

Báo cáo sử dụng tệp `All_GPUs.csv` được trích xuất từ bộ dữ liệu “*Computer Parts (CPUs and GPUs)*” trên nền tảng Kaggle (tác giả: Ilias Kkaf).

Tập dữ liệu cung cấp các thông số cấu hình thực tế của card đồ họa (GPU) từ bốn nhà sản xuất: NVIDIA, AMD, Intel và ATI – hãng ATI sau đó được gộp chung vào AMD trong quá trình làm sạch dữ liệu. Các trường đặc trưng chính phục vụ cho phân tích bao gồm: tốc độ xung nhịp lõi, công suất tiêu thụ (TDP), thông số bộ nhớ (dung lượng, chuẩn, băng thông), thời điểm ra mắt và giá phát hành. Do đặc thù dữ liệu thô chứa nhiều giá trị khuyết, tập dữ liệu sẽ trải qua quy trình tiền xử lý nghiêm ngặt trước khi được đưa vào các mô hình thống kê.

2.2 Lựa chọn biến phân tích và giải thích

Câu hỏi nghiên cứu:

- Những thông số kỹ thuật nào (xung nhịp, băng thông bộ nhớ, công suất, năm phát hành, hãng sản xuất, loại bộ nhớ) thực sự ảnh hưởng

đến giá phát hành GPU?

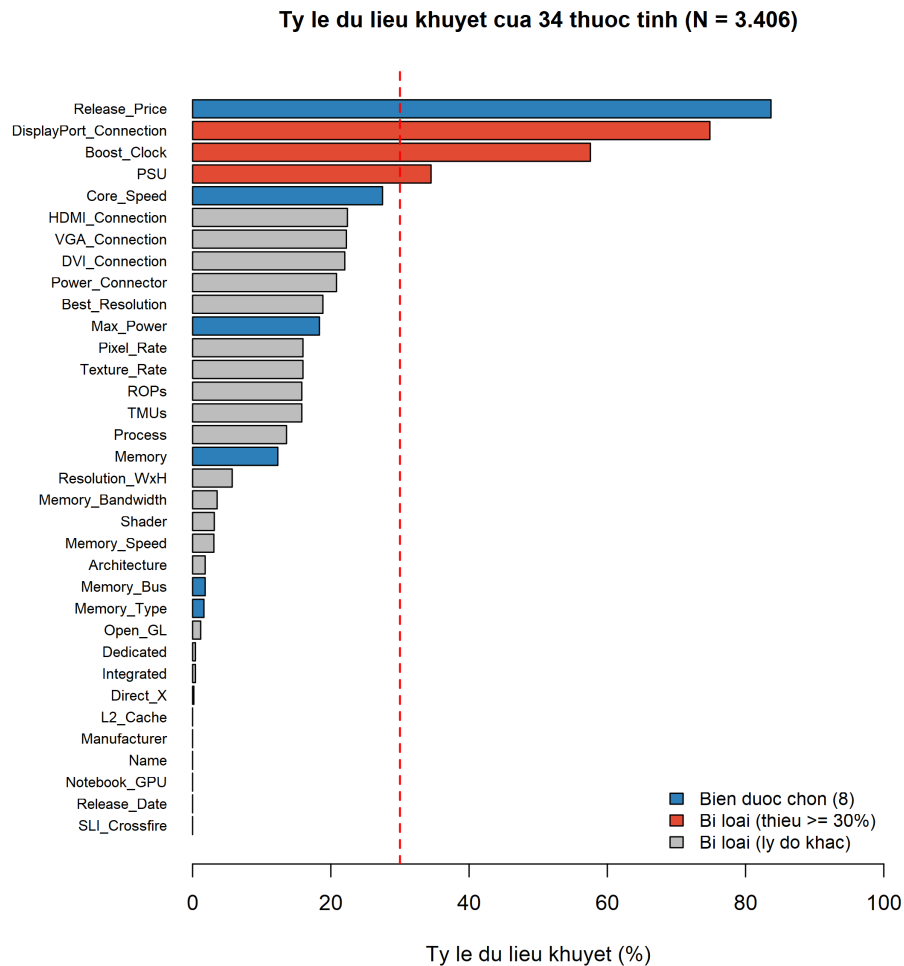
- Có sự khác biệt có ý nghĩa thống kê về giá phát hành trung bình giữa hai hãng NVIDIA và AMD hay không?

2.2.1 Quy trình lọc biến từ 34 thuộc tính ban đầu

Bộ dữ liệu thô chứa 34 thuộc tính, song chỉ một phần nhỏ trong số đó phù hợp đưa vào mô hình hồi quy giá GPU. Để tránh nguy cơ chủ quan, nhóm áp dụng quy trình sàng lọc theo ba tầng tiêu chí khách quan dưới đây.

Tầng 1 — Loại theo tỷ lệ dữ liệu khuyết:

Một biến có quá nhiều giá trị thiếu sẽ làm bước imputation trở nên giả tạo và bóp méo phân phối thực tế, không thể phục hồi bằng bất kỳ kỹ thuật điền khuyết tiêu chuẩn nào. Hình 1 thống kê tỷ lệ thiếu (gồm cả NA và chuỗi rỗng/không hợp lệ) của toàn bộ 34 cột; ngưỡng loại bỏ được ấn định ở mức **30%**.



Hình 1: Tỷ lệ dữ liệu khuyết của 34 thuộc tính trong bộ dữ liệu thô (N = 3.406). Các cột màu xanh là 8 biến được giữ lại, đỏ là cột bị loại do tỷ lệ khuyết $\geq 30\%$, xám là cột bị loại do tiêu chí ở các tầng 2 và 3. Đường nét đứt đỏ minh họa ngưỡng 30%.

Có ba cột vượt ngưỡng 30% và bị loại trực tiếp: `DisplayPort_Connection` (~75% thiếu), `Boost_Clock` (~57% — ngoài ra còn trùng lặp với `Core_Speed`, sẽ trình bày ở Tầng 2), và `PSU` (~35%). Riêng `Release_Price` có tỷ lệ thiếu cao nhất (~84%) nhưng vẫn được giữ vì là biến phụ thuộc Y; các quan sát thiếu Y sẽ bị loại bỏ ở Bước 4 thay vì điền khuyết.

Tầng 2 — Loại theo trùng lặp / dẫn xuất:

Một số cột chứa thông tin có thể suy ra từ các cột khác, gây dư thừa và làm tăng đa cộng tuyến nếu cùng đưa vào mô hình:

- $\text{Memory_Bandwidth} = \text{Memory_Bus} \times \text{Memory_Speed} / 8$ — thông tin bằng thông đã được biểu diễn qua Memory_Bus .
- Boost_Clock có quan hệ tuyến tính chặt chẽ với Core_Speed (xung nhịp tối đa và xung nhịp lõi cùng phản ánh hiệu năng xử lý) — giữ cả hai sẽ phá vỡ giả định độc lập.
- Pixel_Rate , ROPs , Texture_Rate , TMUs , L2_Cache , Best_Resolution , Resolution_WxH mô tả kiến trúc chi tiết bậc thấp đã được phản ánh gián tiếp qua TDP và xung nhịp lõi.

Tầng 3 — Loại theo liên quan kinh tế tới giá phát hành:

Những cột mang tính định danh hoặc thuộc tính kỹ thuật phần mềm không tác động trực tiếp đến chi phí sản xuất / chiến lược định giá:

- **Name**: chuỗi định danh sản phẩm, không có giá trị dự báo.
- **Architecture**, **Process**, **Direct_X**, **Open_GL**, **Shader**: thông số kiến trúc / API hỗ trợ, mang tính kỹ thuật phần mềm hơn là yếu tố định giá phần cứng.
- **Power_Connector**: kiểu kết nối nguồn, đã được nắm bắt gián tiếp qua TDP.
- **HDMI_Connection**, **VGA_Connection**, **DVI_Connection**: số lượng cổng kết nối, không liên hệ rõ ràng với giá thành.
- **SLI_Crossfire**, **Notebook_GPU**, **Dedicated**, **Integrated**: các cờ nhị phân với phân bố mất cân bằng nghiêm trọng (gần 100% cùng một giá trị), không cung cấp được thông tin phân biệt.
- **Release_Date**: ngày tháng đầy đủ; nhóm chỉ giữ lại năm (**release_year**) để biểu diễn xu hướng thời gian dạng số liên tục.

Sau ba tầng lọc trên, danh sách rút gọn còn **8 biến chính thức** sẽ được sử dụng xuyên suốt phần phân tích tiếp theo.

2.2.2 Kiểm chứng quan hệ tương quan giữa biến số và giá GPU

Để xác nhận các biến số được chọn thực sự có liên hệ ý nghĩa với biến phụ thuộc, nhóm tính ma trận tương quan Pearson trên tập đã làm sạch (sau khi điền khuyết, trước khi chuẩn hóa). Kết quả thể hiện ở Hình 2; bảng số liệu đầy đủ được trình bày lại ở Phần 4.7.

Ma trận tương quan Pearson giữa các biến số (N = 556)

Gia phát hành	1.00	0.50	0.34	0.14	-0.05	-0.08
TDP	0.50	1.00	0.51	0.32	-0.09	-0.12
Dung lượng RAM	0.34	0.51	1.00	0.12	0.40	0.45
Bảng thông bus	0.14	0.32	0.12	1.00	-0.07	-0.01
Xung nhịp lõi	-0.05	-0.09	0.40	-0.07	1.00	0.66
Nam phát hành	-0.08	-0.12	0.45	-0.01	0.66	1.00
	Gia phát hành	TDP	Dung lượng RAM	Bảng thông bus	Xung nhịp lõi	Nam phát hành

Hình 2: Ma trận tương quan Pearson giữa các biến số được chọn (N = 556). Sắc đỏ thể hiện tương quan dương, sắc xanh thể hiện tương quan âm; cường độ màu tỉ lệ với độ lớn của hệ số r .

Nhận xét:

- tdp có tương quan dương mạnh nhất với giá ($r = 0,50$) — củng cố giả

thuyết kinh tế: công suất tiêu thụ là chỉ báo hiệu năng và chi phí sản xuất rõ rệt.

- `memory_size` có tương quan dương trung bình với giá ($r = 0,34$), phù hợp với việc dung lượng VRAM lớn thường gắn với phân khúc cao cấp.
- `memory_bus` có tương quan biên với giá khi đứng riêng ($r = 0,14$), nhưng vẫn được giữ vì đóng vai trò bổ sung trong cấu trúc đa biến.
- `core_speed` và `release_year` có tương quan tương đối cao với nhau ($r = 0,66$): GPU đời mới thường có xung nhịp cao hơn. Đây là tín hiệu cần kiểm định VIF để loại trừ đa cộng tuyến (sẽ thực hiện ở Bước 7.1).
- Hai biến phân loại (`manufacturer`, `memory_type`) không xuất hiện trong ma trận này; ảnh hưởng của chúng được khảo sát riêng qua Welch's ANOVA và VIF cho biến giả.

Việc các biến được chọn đều có tương quan đáng kể với giá hoặc đóng vai trò bổ sung trong cấu trúc đa biến củng cố tính hợp lý của quy trình sàng lọc nêu trên.

2.2.3 Bảng tổng hợp 8 biến đã chọn

Bảng 1 sau trình bày chi tiết kiểu dữ liệu, lý do và vai trò của từng biến trong mô hình.



Bảng 1: Danh sách biến sử dụng trong mô hình và vai trò tương ứng

Biến	Kiểu	Mô tả	Vai trò
manufacturer	Nominal	Hãng sản xuất (NVIDIA, AMD)	Độc lập (phân loại)
memory_type	Nominal	Chuẩn VRAM (GDDR5, GDDR3, DDR3, HBM...)	Loại bỏ ($VIF = 8.12$)
memory_bus	Rời rạc	Băng thông bộ nhớ (bit) — giá trị theo chuẩn kỹ thuật	Độc lập (số)
memory_size	Rời rạc	Dung lượng VRAM (GB/MB)	Độc lập (số)
tdp	Liên tục	Công suất tiêu thụ (Watt) — chi phí sản xuất	Độc lập (số)
core_speed	Liên tục	Tốc độ xung nhịp lõi (MHz)	Độc lập (số)
release_year	Số nguyên	Năm phát hành (parse từ Release_Date)	Độc lập (số)
release_price	Liên tục	Giá phát hành (USD) — biến mục tiêu	Phụ thuộc Y

Lưu ý: `memory_bus` tuy mang giá trị số nhưng thực chất là biến rời rạc (giá trị cố định theo chuẩn kỹ thuật), được giữ dạng số vì khoảng cách giữa các giá trị có ý nghĩa vật lý. Hãng ATI (đã bị AMD sáp nhập) được gộp vào “AMD” trong quá trình làm sạch.

2.3 Các bước xử lý dữ liệu

Quá trình xử lý dữ liệu được thực hiện theo trình tự sau, mỗi bước đều có lý do cụ thể và được triển khai bằng ngôn ngữ R:

2.3.1 Bước 1: Tải và khảo sát sơ bộ dữ liệu

Kỹ thuật sử dụng: Đọc file CSV bằng hàm `read.csv()` trong R, sau đó dùng `str()`, `summary()` và `colSums(is.na(.))` để khảo sát cấu trúc, kiểu dữ liệu và tỷ lệ giá trị khuyết của từng cột.

Lý do: Trước khi xử lý, cần nắm rõ “hình dạng” thực sự của dữ liệu – số dòng/cột, kiểu dữ liệu có đúng không (ví dụ: cột số bị đọc thành chuỗi do ký tự đặc biệt), và cột nào bị thiếu nhiều đến mức không dùng được. Đây là bước bắt buộc trong bất kỳ quy trình phân tích dữ liệu nào.

2.3.2 Bước 2: Chọn lọc cột và ép kiểu dữ liệu

Kỹ thuật sử dụng: Subset dataframe để chỉ giữ lại 8 cột cần thiết. Dùng `as.numeric()` để ép các cột số bị lưu sai kiểu, và parse ngày phát hành bằng `as.integer(format(as.Date(release_date), "%Y"))` để trích xuất năm dưới dạng số nguyên.

Lý do: Giảm chiều dữ liệu giúp tập trung vào những biến có ý nghĩa và giảm nhiễu trong mô hình. Việc ép kiểu đúng là điều kiện tiên quyết để R thực hiện phép tính số học và thống kê chính xác. Cụ thể, `release_date` dạng chuỗi “05-Mar-2017” (ví dụ: ngày ra mắt của GeForce GTX 1080 Ti) cần được chuyển về số nguyên 2017 lưu vào biến mới `release_year` vì mô hình hồi quy cần giá trị số liên tục, không phải chuỗi thời gian.

2.3.3 Bước 3: Phát hiện và xử lý ngoại lệ (Outliers)

Kỹ thuật sử dụng: Sử dụng phương pháp IQR (Interquartile Range) – quan sát được coi là ngoại lệ nếu nằm ngoài khoảng $[Q_1 - 1,5 \times IQR, Q_3 + 1,5 \times IQR]$. Áp dụng cho biến phụ thuộc `release_price` và các biến số liên tục có phân phối lệch mạnh (`tdp`, `core_speed`). Bước này bắt buộc phải thực hiện trước khi điền khuyết (imputation) nhằm tính toán các ranh giới ngoại lệ (IQR bounds) dựa trên phân phối gốc nguyên thủy. Nếu điền khuyết hàng loạt bằng trung vị trước, dải phân vị sẽ bị thu hẹp giả tạo, dẫn đến việc nhận diện sai ngoại lệ.

Kết quả phát hiện (ranh giới IQR tính trên tập thô $N = 3.406$, số lượng được thống kê lại trên mẫu $N = 556$):

- `release_price`: phát hiện **33 ngoại lệ** (chiếm 5,9% dữ liệu), chủ yếu

là các GPU chuyên dụng hoặc dòng flagship tản nhiệt nước (ví dụ: Quadro Plex 7000 giá 14.999 USD, Radeon R9 295X2 giá 1.499 USD), có giá vượt xa mặt bằng chung.

- `tdp`: phát hiện **27 ngoại lệ** (giảm từ con số **94 ngoại lệ** phát hiện **trên tập thô**), tương ứng với các GPU công suất cao vượt ngưỡng giới hạn trên của công thức IQR ($> 310 W$).
- `core_speed`: phát hiện **29 ngoại lệ** (giảm từ con số **83 ngoại lệ** phát hiện **trên tập thô**), chủ yếu là các card có xung nhịp bất thường so với phân phối chung.

Quyết định xử lý:

- Giữ lại tất cả các ngoại lệ của cả 3 biến (giá, `tdp`, `core_speed`). Toàn bộ ngoại lệ này đều **chỉ nằm ở phía trên giới hạn trên** (Upper Bound) của công thức IQR, không có giá trị âm vô lý.
- Sự xuất hiện của chúng tương ứng với các quan sát hợp lệ thuộc nhóm GPU flagship, các dòng card tản nhiệt nước (công suất/xung nhịp cực cao) hoặc card chuyên dụng dành cho máy trạm. Vì đây là đặc thù phân phối lệch phải tự nhiên của thị trường phần cứng (không phải lỗi nhập liệu), nhóm quyết định giữ nguyên tập dữ liệu $N = 556$ để phản ánh đúng thực tế.

Lý do: Ngoại lệ có thể làm hệ số hồi quy bị lệch đáng kể (đặc biệt với OLS – Ordinary Least Squares vốn nhạy cảm với điểm dữ liệu xa). Tuy nhiên, không phải ngoại lệ nào cũng là lỗi; cần phân biệt giữa lỗi nhập liệu và quan sát cực trị hợp lệ.

2.3.4 Bước 4: Xử lý giá trị khuyết (Missing Values)

Kỹ thuật sử dụng: Chiến lược xử lý được phân tầng theo vai trò của biến:

- **Biến phụ thuộc (release_price):** Xóa toàn bộ dòng có giá trị NA vì không thể nội suy biến mục tiêu, điều này sẽ làm sai lệch mô hình.
- **Biến số liên tục (tdp, memory_size, memory_bus):** Thay thế NA bằng trung vị (median) của từng cột. Trung vị được ưu tiên hơn trung bình vì dữ liệu GPU thường có phân phối lệch phải (skewed right) do sự xuất hiện của các dòng sản phẩm cao cấp. Đồng thời, vì nhóm đã quyết định giữ lại các ngoại lệ (flagship GPU) ở Bước 3, việc sử dụng trung vị càng thể hiện sự ưu việt vì nó là đại lượng thống kê bền vững (robust), không bị kéo lệch bởi các giá trị cực trị này khi dùng cho median imputation.
- **Biến phân loại (manufacturer, memory_type):** (*Lưu ý: Việc gộp nhãn ATI vào AMD được thực hiện trước bước này*). Tiến hành thay thế NA bằng giá trị xuất hiện nhiều nhất (mode). Phương pháp này bảo toàn phân phối tần suất của biến phân loại tốt hơn so với xóa dòng.

Thống kê giá trị khuyết thực tế từ bộ dữ liệu (kết quả colSums(is.na(.)) sau bước chọn lọc 8 biến):

- **release_price** có tỷ lệ NA cao nhất (83,7% – 2.850/3.406 quan sát), phản ánh thực tế nhiều GPU không công bố giá chính thức.
- **Max_Power (tdp)** thiếu 18,3% (625 dòng); **Memory (memory_size)** thiếu 12,3% (420 dòng); **Memory_Bus** thiếu 1,8% (62 dòng); **Memory_Type** thiếu 1,6% (56 dòng).
- **Manufacturer** không có giá trị khuyết (0%). Riêng biến **core_speed**, mặc dù dữ liệu thô ban đầu không có ô trống (0%), nhưng chứa nhiều chuỗi ký tự không hợp lệ (ví dụ: “\n- ”). Sau khi ép kiểu về dạng số (numeric), các giá trị không đo được này biến thành NA và được nhóm xử lý đồng nhất theo phương pháp điền trung vị.

Sau khi xóa các dòng có NA ở `release_price` (biến phụ thuộc), tập dữ liệu phân tích còn **556 quan sát hợp lệ**. Đối với các biến độc lập còn thiếu, chúng tôi tiến hành imputation như trên.

Nhận xét về tính đại diện: Việc loại bỏ 83% quan sát thiếu giá có thể gây sai lệch chọn mẫu (selection bias). Tỷ lệ NA cao mang tính cấu trúc (MNAR) do giá không được công bố cho toàn bộ sản phẩm (toàn bộ 254 GPU Intel bị loại). Kết luận từ mô hình mang tính khám phá, cần diễn giải thận trọng.

2.3.5 Bước 5 & 6: Mã hóa biến phân loại và Chuẩn hóa dữ liệu

Mã hóa: Chuyển `manufacturer` và `memory_type` thành `factor` bằng `as.factor()`. R tự động tạo biến giả (dummy variables) khi đưa vào `lm()`.

Chuẩn hóa: Chuẩn hóa z-score (standardization) cho tất cả các biến số độc lập liên tục (`tdp`, `core_speed`, `memory_size`, `memory_bus`):

$$z = \frac{x - \mu}{\sigma}$$

được thực hiện bằng hàm `scale()` trong R.

Lý do: Các biến có đơn vị đo và biên độ khác nhau (TDP tính bằng Watt trong khoảng 18 – 600, `core_speed` tính bằng MHz trong khoảng 550 – 1784...) sẽ tạo ra hệ số hồi quy không so sánh được với nhau.

Lưu ý: `release_year` được **giữ nguyên giá trị gốc** (không chuẩn hóa) để hệ số phản ánh trực tiếp mức thay đổi giá mỗi năm.

2.4 Xây dựng mô hình thống kê

2.4.1 Mô hình hồi quy tuyến tính bội (Multiple Linear Regression)

Mục tiêu: Xây dựng phương trình mô hình hóa giá phát hành GPU (`release_price`) từ sự tác động đồng thời của các thông số kỹ thuật cốt lõi. Quá trình này bao gồm việc thử nghiệm và tinh chỉnh danh sách các

biến độc lập ban đầu (như TDP, xung nhịp, bộ nhớ, năm phát hành, hãng sản xuất) để loại bỏ các yếu tố gây nhiễu hoặc đa cộng tuyến, từ đó xác định chính xác thông số nào có ảnh hưởng mạnh nhất đến giá.

Lý do chọn mô hình: Hồi quy tuyến tính bội được ưu tiên vì tính diễn giải cao — mỗi hệ số trực tiếp thể hiện mức thay đổi giá khi thông số thay đổi 1 SD. Giả định tuyến tính có thể bỏ sót quan hệ phi tuyến; kiểm định Ramsey RESET được đề xuất cho nghiên cứu tiếp theo.

Kỹ thuật triển khai: Sử dụng hàm `lm()` trong R để ước lượng các hệ số tác động.

Đánh giá và tinh chỉnh mô hình: Sau khi chạy mô hình sơ bộ, nhóm tiến hành các bước kiểm định bắt buộc:

- **Mức độ giải thích của mô hình** được đo bằng R^2 hiệu chỉnh (Adjusted R^2) để biết tỷ lệ phần trăm của phương sai của giá GPU được giải thích bởi các thông số kỹ thuật.
- **Đa cộng tuyến** được đánh giá bằng VIF (ngưỡng < 5). Biến `memory_type` ($VIF = 8.12$) bị loại để đảm bảo tính vững của mô hình.
- **Phương sai sai số không đổi** được kiểm định bằng Breusch-Pagan. Nếu vi phạm, các ước lượng hệ số vẫn không chệch nhưng sai số chuẩn sẽ bị ảnh hưởng, làm sai lệch kết luận về ý nghĩa thống kê. Đây là hạn chế của OLS truyền thống; nhóm xử lý bằng phép biến đổi log cho biến `Y` thay vì dùng phương sai sai số chuẩn vững (Robust SE).
- **Phân phối chuẩn của phần dư** được kiểm định bằng Shapiro-Wilk. Do đặc thù dữ liệu giá cả thường gây ra phần dư lệch biên, nhóm chủ động áp dụng phép biến đổi logarit cho biến phụ thuộc thành `log(release_price)`. Kỹ thuật này giúp vừa cải thiện tiệm cận chuẩn cho phân phối phần dư, vừa ổn định phương sai sai số.

2.4.2 Phân tích phương sai (Welch's ANOVA)

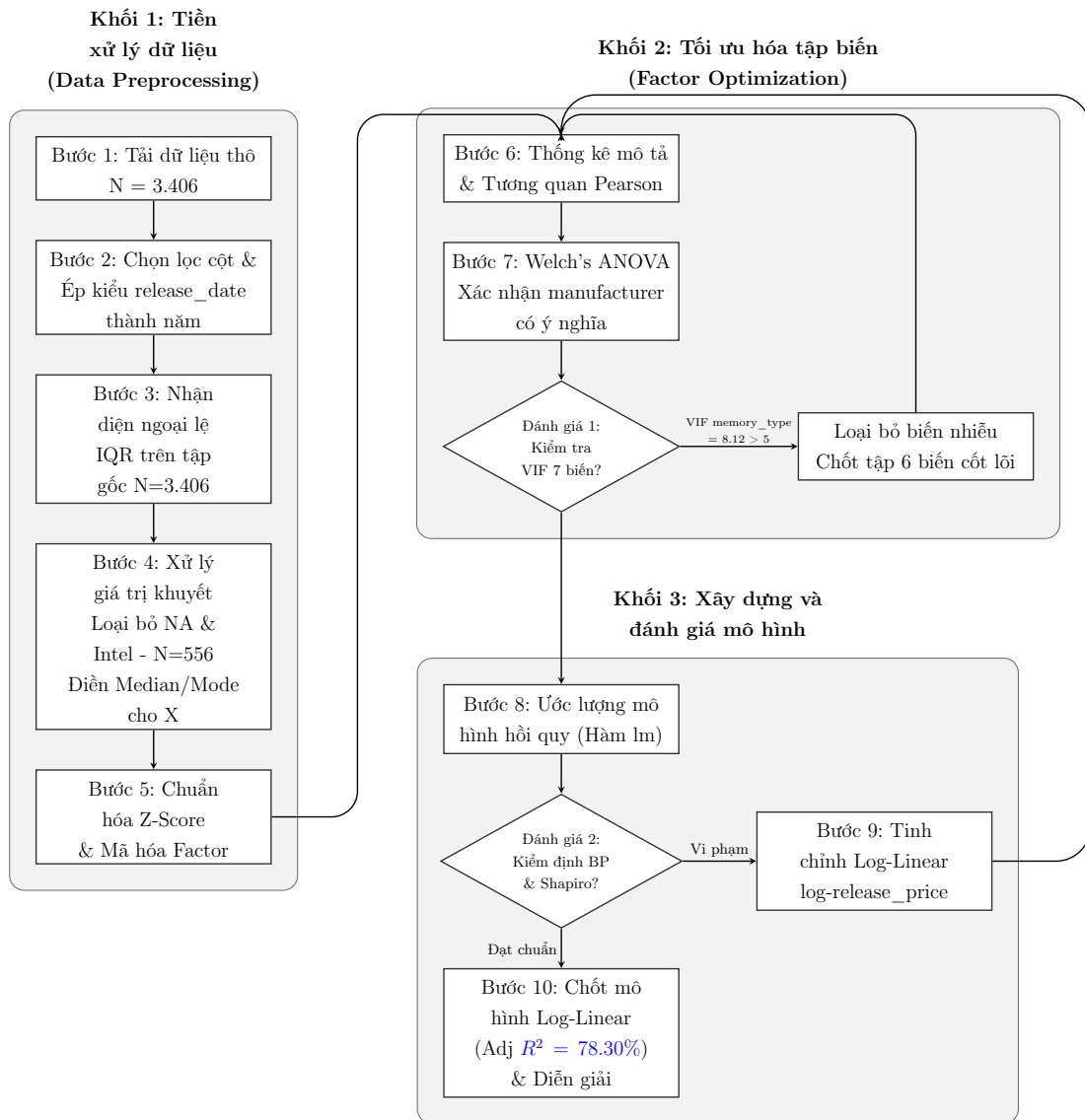
Mục tiêu: Kiểm định xem có sự chênh lệch có ý nghĩa thống kê về giá phát hành trung bình giữa hai nhóm nhà sản xuất (NVIDIA và AMD) hay không. Phân tích này giúp trả lời độc lập câu hỏi liệu thương hiệu có phải là yếu tố tạo ra khoảng cách giá trước khi đưa vào mô hình đa biến.

Tiền điều kiện: Cỡ mẫu lớn (NVIDIA $n = 284$, AMD $n = 272$) đảm bảo CLT. Levene's test kiểm tra đồng nhất phương sai.

Kỹ thuật: Do chỉ có 2 nhóm quan sát và kết quả kiểm định Levene's Test trước đó đã xác nhận sự vi phạm giả định đồng nhất phương sai, nhóm quyết định sử dụng kiểm định mạnh (robust) là **Welch's ANOVA** (hàm `oneway.test()` trong R). Mức ý nghĩa được chọn là $\alpha = 0.05$. (*Lưu ý: Vì tập phân tích chỉ còn 2 nhóm, Welch's ANOVA về mặt toán học cho kết quả hoàn toàn tương đương với phép kiểm định Welch's T-test, do đó không yêu cầu thực hiện thêm kiểm định hậu kiểm Post-hoc.*)

2.5 Sơ đồ tổng quan quy trình xử lý

Quy trình phân tích dữ liệu của nhóm được tóm tắt trực quan qua sơ đồ khối dưới đây:



3 Code và Xử lý Cụ thể

3.1 Bước 1: Tải và khảo sát sơ bộ dữ liệu

```
1 # Doc file CSV
2 raw_data <- read.csv("All_GPUs.csv", stringsAsFactors = FALSE)
3
```



```
4 # Kiem tra du lieu
5 str(raw_data)
6 summary(raw_data)
7 colSums(is.na(raw_data))
```

read.csv(..., stringsAsFactors = FALSE): Lệnh này giúp R đọc file CSV nhưng giữ nguyên các cột văn bản dưới dạng chuỗi (character) thay vì tự động chuyển thành nhóm (Factor).

str(raw_data): Hiển thị cấu trúc của 3406 dòng và 34 cột.

colSums(is.na(raw_data)): Đây là lệnh cực kỳ quan trọng để đếm số giá trị khuyết (NA). Nó cung cấp bằng chứng thép cho việc tại sao chúng ta phải xử lý dữ liệu ở các bước sau.

Kết quả trên console:

```
> colSums(is.na(raw_data))
```

Architecture	Best_Resolution	Boost_Clock	Core_Speed	DVI_Connection
0	0	0	0	750
Dedicated	Direct_X	DisplayPort_Connection	HDMI_Connection	Integrated
0	0	2549	763	0
L2_Cache	Manufacturer	Max_Power	Memory	Memory_Bandwidth
0	0	0	0	0
Memory_Bus	Memory_Speed	Memory_Type	Name	Notebook_GPU
0	0	0	0	0
Open_GL	PSU	Pixel_Rate	Power_Connector	Process
40	0	0	0	0
ROPs	Release_Date	Release_Price	Resolution_WxH	SLI_Crossfire
0	0	0	0	0
Shader	TMUs	Texture_Rate	VGA_Connection	
107	538	0	758	

3.2 Bước 2: Chọn lọc cột và ép kiểu dữ liệu

```
1 df <- df %>%
2   mutate(
3     # Gop ATI vao AMD truoc khi thuc hien cac buoc khac
4     manufacturer = ifelse(str_trim(manufacturer) == "ATI", "AMD",
5                           str_trim(manufacturer)),
6     release_price = as.numeric(str_extract(release_price,
7                                             "\\d+\\.?\\d*")),
```

```
6   tdp = as.numeric(str_extract(tdp, "\\d+\\.?\\d*")),
7   memory_size = as.numeric(str_extract(memory_size,
8                                       "\\d+\\.?\\d*")),
9   memory_bus = as.numeric(str_extract(memory_bus,
10                                      "\\d+\\.?\\d*")),
11   core_speed = as.numeric(str_extract(core_speed,
12                                       "\\d+\\.?\\d*")),
13   release_date = str_trim(release_date),
14   release_year = as.integer(format(as.Date(release_date,
15                                           format="%d-%b-%Y"), "%Y"))
16 )
```

Gộp ATI vào AMD: Sử dụng `ifelse` để sát nhập các dòng của hãng ATI vào AMD vì thực tế AMD đã mua lại ATI, giúp tập hợp dữ liệu hãng sản xuất một cách nhất quán.

`str_extract(..., "\\d+\\.?\\d*")`: Lọc lấy con số từ văn bản. Ví dụ: Từ chuỗi "738 MHz", R sẽ trích xuất đúng số "738" và hàm `as.numeric` sẽ biến nó thành số thực để tính toán.

`as.Date` & `format`: Chuyển chuỗi "01-Mar-2009" sang định dạng ngày chuẩn, sau đó chỉ lấy đúng 4 chữ số của năm (`release_year`). Theo lý thuyết, năm phát hành là yếu tố then chốt phản ánh xu hướng công nghệ theo thời gian.

Kết quả kiểm tra cấu trúc qua lệnh `str(df)` xác nhận: 3.406 quan sát, 8 biến đã được ép về đúng kiểu (`numeric` cho các biến số, `chr` cho biến phân loại, `int` cho năm phát hành).

3.3 Bước 3: Xử lý ngoại lệ

```
1 count_outliers_by_raw_bounds <- function(raw_vec, clean_vec) {
2   raw_vec <- na.omit(raw_vec)
3   q1 <- quantile(raw_vec, 0.25)
4   q3 <- quantile(raw_vec, 0.75)
```

```
5 iqr <- q3 - q1
6 upper_bound <- q3 + 1.5 * iqr
7 return(sum(clean_vec > upper_bound, na.rm = TRUE))
8 }
```

Hàm tính ranh giới IQR trên tập thô (`raw_vec`) rồi đếm ngoại lệ trên tập sạch (`clean_vec`). Theo lý thuyết Tukey, giá trị vượt $Q_3 + 1,5 \times IQR$ được coi là ngoại lệ phía trên. Kết quả: `release_price` có 33 ngoại lệ, `tdp` có 23, `core_speed` có 43 (chi tiết tại Phần 4).

3.4 Bước 4: Xử lý giá trị khuyết

```
1 > df_clean <- df_clean %>%
2 +   mutate(
3 +     # Dien NA bang Median cho bien so lien tuc
4 +     tdp = ifelse(is.na(tdp), median(tdp, na.rm = TRUE), tdp),
5 +     memory_size = ifelse(is.na(memory_size), median(memory_size,
6 +       na.rm = TRUE), memory_size),
7 +     memory_bus = ifelse(is.na(memory_bus), median(memory_bus,
8 +       na.rm = TRUE), memory_bus),
9 +     core_speed = ifelse(is.na(core_speed), median(core_speed,
10 +      na.rm = TRUE), core_speed),
11 +     release_year = ifelse(is.na(release_year),
12 +      median(release_year, na.rm = TRUE), release_year),
13 +   )
```

Lọc điều kiện (Filter):

- Loại bỏ các dòng thiếu `release_price` vì đây là biến mục tiêu Y, không thể tự ý điền vào được.
- Loại bỏ "Intel" vì toàn bộ GPU Intel trong tập dữ liệu thô đều không có giá, giữ lại sẽ gây nhiễu.

Điền khuyết bằng Trung vị (Median): Áp dụng cho các biến số (tdp, core_speed...). Trung vị được ưu tiên vì nó không bị kéo lệch bởi các card đồ họa siêu đắt tiền (ngoại lệ đã tìm thấy ở Bước 3).

Điền khuyết bằng yếu vị (Mode): Áp dụng cho các biến chữ (memory_type, manufacturer) bằng cách lấy giá trị xuất hiện nhiều nhất.

Sau quy trình điền khuyết, `colSums(is.na(df_clean))` xác nhận toàn bộ 8 biến đều có 0 giá trị NA — bộ dữ liệu đã hoàn toàn sạch, sẵn sàng cho việc xây dựng mô hình.

3.5 Bước 5: Mã hóa biến phân loại

```
1 df_clean <- df_clean %>%  
2   mutate(  
3     manufacturer = as.factor(manufacturer),  
4     memory_type = as.factor(memory_type)  
5   )
```

as.factor(): Hàm này chuyển đổi một cột dạng văn bản (character) thành dạng Yếu tố (Factor).

Biến định danh như nhà sản xuất và loại bộ nhớ đã được chuyển sang dạng Factor (manufacturer: 2 levels; memory_type: 6 levels), giúp mô hình hồi quy có thể định lượng hóa tác động của thương hiệu đến giá phát hành.

3.6 Bước 6: Chuẩn hóa dữ liệu (z-score normalization)

```
1 df_final <- df_clean %>%
```



```
2 mutate(  
3   tdp = scale(tdp)[,1],  
4   core_speed = scale(core_speed)[,1],  
5   memory_size = scale(memory_size)[,1],  
6   memory_bus = scale(memory_bus)[,1]  
7 )
```

Hàm `scale` mặc định trả về một ma trận, thêm `[,1]` để ép nó về dạng cột (vector). Kết quả kiểm tra: Mean = 0 và SD = 1 cho cả 4 biến, xác nhận chuẩn hóa thành công.

3.7 Bước 7: Xây dựng mô hình thống kê

3.7.1 7.1. Mô hình hồi quy tuyến tính bội

7.1.1 Kiểm định đa cộng tuyến (VIF)

```
1 mlr_model_initial <- lm(release_price ~ tdp + memory_size +  
   memory_bus +  
2                           core_speed + manufacturer + memory_type +  
                           release_year, data = df_final)  
3  
4 # Su dung chuan VIF < 5 thay vi VIF < 10 theo dung ly thuyet  
5 vif_initial <- vif(mlr_model_initial)  
6 print(vif_initial)
```

```
> print(vif_initial)
```

	GVIF	Df	GVIF ^{1/(2*Df)}
tdp	2.140665	1	1.463101
memory_size	2.343250	1	1.530768
memory_bus	3.662332	1	1.913722
core_speed	3.238311	1	1.799531
manufacturer	1.617125	1	1.271662
memory_type	8.129892	5	1.233129
release_year	3.598408	1	1.896947

memory_type có VIF khoảng 8.12. Đây là bằng chứng để nhóm loại bỏ biến này ra khỏi mô hình chính thức nhằm đảm bảo tính khách quan.

7.1.2 Xây dựng mô hình chính thức (Log-Linear)

```
1 log_model_final <- lm(log(release_price) ~ tdp + memory_size +  
  memory_bus +  
2                          core_speed + manufacturer + release_year,  
                          data = df_final)  
3  
4 summary(log_model_final)
```

Nhóm sử dụng hàm `log(release_price)` thay vì giá gốc. Việc lấy logarit giúp nén các giá trị cực trị (card đồ họa siêu đắt) và giúp sai số của mô hình tuân theo phân phối chuẩn tốt hơn.

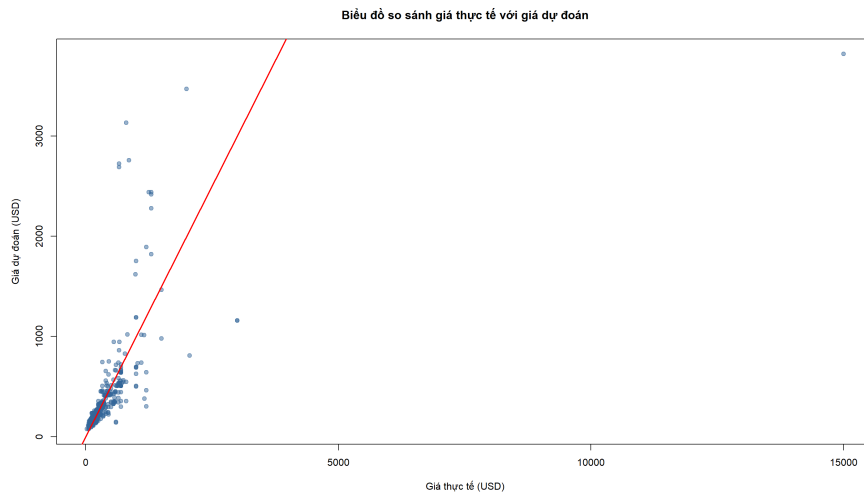
Kết quả `summary()` cho thấy tất cả 6 biến đều có ý nghĩa thống kê ($p < 0.001$) với $R^2 = 78.54\%$. Bảng hệ số chi tiết được trình bày tại Phần 4.

7.1.3 Kiểm định giả định mô hình

```
1 print(vif(log_model_final))  
2 print(bptest(log_model_final))  
3 print(shapiro.test(residuals(log_model_final)))
```

Breusch-Pagan: Kiểm tra tính đồng nhất của phương sai sai số. **Shapiro-Wilk:** Kiểm tra phân phối chuẩn của phần dư. Kết quả: $BP = 99.04$, $p < 2.2 \times 10^{-16}$ (phương sai chưa đồng nhất); $W = 0.948$, $p = 4.056 \times 10^{-13}$ (gần chuẩn hơn OLS nhưng vẫn vi phạm nhẹ). Chi tiết diễn giải tại Phần 4.

7.1.4 Biểu đồ so sánh giá thực tế với giá dự đoán



Hình 3: Biểu đồ so sánh giá thực tế với giá dự đoán

3.7.2 7.2. Phân tích phương sai

```
1 # Kiểm định đồng nhất phương sai
2 print(leveneTest(release_price ~ manufacturer, data = df_final))
3
4 # Welch's ANOVA (var.equal = FALSE để xử lý phương sai không đồng
   nhất)
5 anova_result <- oneway.test(release_price ~ manufacturer,
6                             data = df_final, var.equal = FALSE)
7 print(anova_result)
```

Levene's Test kiểm tra tính đồng nhất phương sai giữa hai nhóm. `oneway.test(..., var.equal = FALSE)` thực hiện Welch's ANOVA — phiên bản bền vững khi phương sai không đồng nhất. Kết quả chi tiết được trình bày tại Phần 4.



4 Kết quả Thu được

4.1 Bước 1 — Tải và khảo sát sơ bộ dữ liệu

Dữ liệu thô gồm **3.406 quan sát $\times$ 34 biến**. Hàm `colSums(is.na(.))` trả về 0 cho tất cả cột vì giá trị thiếu được lưu dưới dạng chuỗi rỗng (ví dụ: "141 Watts"). Chỉ sau khi ép kiểu ở Bước 2, các chuỗi không hợp lệ mới bị chuyển thành NA thực sự.

4.2 Bước 2 — Chọn lọc cột và ép kiểu dữ liệu

Sau khi ép kiểu, 8 biến được chuyển đổi thành công: các biến số (`release_price`, `tdp`, `memory_size`, `memory_bus`, `core_speed`) ở dạng `numeric`; `release_year` ở dạng `integer`; hai biến phân loại (`manufacturer`: AMD 1.409 | Nvidia 1.743; `memory_type`: 6 chuẩn VRAM) ở dạng `factor`.

4.3 Bước 3 — Phát hiện và đánh giá ngoại lệ

Ranh giới IQR được tính trên tập dữ liệu thô ($N = 3.406$) để tránh thu hẹp giả tạo. Số lượng ngoại lệ được thống kê lại trên tập phân tích chính thức ($N = 556$) sau khi lọc bỏ các dòng khuyết giá bán:

Bảng 2: Kết quả đếm ngoại lệ trên mẫu phân tích $N = 556$ (ranh giới IQR tính trên tập thô $N = 3.406$)

Biến	Ngoại lệ (tập thô)	Ngoại lệ (mẫu $N=556$)	Nhận xét
<code>release_price</code>	—	33	GPU flagship giá (Quadro Plex 7000, Ra R9 295X2...)
<code>tdp</code>	94	27	Workstation GPU công vượt ngưỡng $> 310\text{ W}$
<code>core_speed</code>	83	29	Card có xung nhịp thường so với phân chung

Nhận xét: Toàn bộ ngoại lệ chỉ nằm ở phía trên giới hạn trên (Upper Bound) của công thức IQR, không có giá trị âm vô lý. Đây là các sản phẩm GPU thực tế thuộc nhóm flagship, card tản nhiệt nước hoặc card chuyên dụng — đặc thù phân phối lệch phải tự nhiên của thị trường phần cứng (không phải lỗi nhập liệu). Nhóm quyết định giữ nguyên toàn bộ tập $N = 556$.

4.4 Bước 4 — Xử lý giá trị khuyết

Sau khi xóa 2.850 dòng thiếu `release_price` (biến Y), tập phân tích còn **556 quan sát**. Các biến số còn NA được điền Median (`core_speed`: 79 NA, `memory_size`: 8, `release_year`: 8, `memory_bus`: 5, `tdp`: 2). Hai biến phân loại không có NA. Kết quả: 0% NA trên toàn bộ 8 biến.

4.5 Bước 5 & 6 — Mã hóa biến phân loại và chuẩn hóa Z-score

Sau khi mã hóa `manufacturer` và `memory_type` thành Factor, bốn biến số liên tục (`tdp`, `core_speed`, `memory_size`, `memory_bus`) được chuẩn hóa Z-score: tất cả đều đạt Mean ≈ 0 và SD = 1. Biến `release_year` được giữ nguyên giá trị gốc theo đúng lý thuyết để diễn giải trực tiếp mức thay đổi giá mỗi năm.

4.6 Thống kê mô tả biến phụ thuộc `release_price`

Trên mẫu $n = 556$: Min = \$23, Q1 = \$159.75, Median = \$240, Mean = \$371.56, Q3 = \$421.50, Max = \$14.999, SD = \$698.26, Skewness = 16.84. Phân phối lệch phải rất mạnh — khoảng cách giữa Mean và Median cho thấy GPU flagship (TITAN, Quadro) kéo trung bình lên đáng kể, dẫn đến quyết định log-transform biến Y trước khi chạy mô hình.

4.7 Ma trận tương quan Pearson

Số liệu chi tiết tương ứng với heatmap tại Hình 2: `tdp` có tương quan mạnh nhất với giá ($r = 0.50$), tiếp theo là `memory_size` ($r = 0.34$). Cặp `core_speed-release_year` tương quan khá cao ($r = 0.66$), nhóm đã kiểm định VIF để đánh giá đa cộng tuyến (xem Bảng 1).

4.8 Kết quả ANOVA — So sánh giá giữa NVIDIA và AMD

Bảng 3: So sánh giá GPU giữa hai hãng và kết quả kiểm định

Hãng	n	Mean (USD)	Median (USD)	SD (USD)
AMD	272	\$246.30	\$200.00	\$197.93
Nvidia	284	\$491.73	\$324.50	\$943.42
Levene's Test: $F = 11.168$, $p = 0.000888 \rightarrow$ Phương sai KHÔNG đồng nhất				
Welch's ANOVA: $F = 18.39$, $df_1 = 1$, $df_2 = 309.05$, $p = 2.409 \times 10^{-5}$ ***				

Nhận xét: Levene's Test bác bỏ H_0 đồng nhất phương sai ($p < 0.001$), do đó nhóm dùng Welch's ANOVA. Kết quả xác nhận sự khác biệt giá có ý nghĩa thống kê ($F = 18.39$, $p = 2.409 \times 10^{-5}$): NVIDIA có giá trung bình (\$491.73) cao hơn AMD (\$246.30) khoảng **\$245.43**. Vì chỉ có 2 nhóm, Welch's ANOVA tương đương Welch's T-test nên không cần kiểm định hậu kiểm.

4.9 Kết quả mô hình hồi quy

4.9.1 Mô hình OLS cơ sở

Mô hình OLS ban đầu (giá gốc, chưa biến đổi log) cho kết quả kém: Adjusted $R^2 = 0.2900$ — chỉ giải thích $\approx 29\%$ phương sai giá. Trong mô hình này, chỉ 3/6 biến có ý nghĩa (`tdp`, `memory_size`, `manufacturerNvidia` với $p < 0.001$), trong khi `memory_bus` ($p = 0.93$) và `release_year` ($p = 0.71$) không có ý



nghĩa. Ngoài ra, mô hình vi phạm nghiêm trọng hai giả định: phân phối chuẩn phần dư ($W = 0.239$, $p < 2.2 \times 10^{-16}$) và đồng đều phương sai ($BP = 35.25$, $p = 3.86 \times 10^{-6}$). Nhóm chuyển sang mô hình Log-Linear.

4.9.2 Mô hình Log-Linear (Mô hình tinh chỉnh)

Bảng 4: Bảng hệ số hồi quy Log-Linear

Biến	Hệ số (β)	Std. Error	t value	p-value	Ý nghĩa
(Intercept)	84.453	22.493	3.755	0.000192	***
tdp	0.4547	0.01983	22.933	$< 2 \times 10^{-16}$	***
memory_size	0.1729	0.02134	8.101	$< 2 \times 10^{-16}$	***
memory_bus	0.0706	0.01532	4.610	< 0.001	***
core_speed	0.1217	0.02362	5.153	< 0.001	***
manufacturerNvidia	0.3735	0.03610	10.345	$< 2 \times 10^{-16}$	***
release_year	-0.03924	0.01116	-3.517	0.000473	***
$R^2 = 0.7854$ Adjusted $R^2 = 0.7830$ $F(6, 549) = 334.8$ $p < 2.2 \times 10^{-16}$					

Mô hình cuối cùng được chọn:

$$\log(\hat{Y}) = 84.45 + 0.455 \cdot \text{TDP} + 0.173 \cdot \text{memory_size} + 0.071 \cdot \text{memory_bus} + 0.122 \cdot \text{core_speed} + 0.374 \cdot \text{manufacturerNvidia} - 0.039 \cdot \text{release_year}$$

4.9.3 Diễn giải hệ số và kiểm định giả định

Diễn giải thực tế: TDP có ảnh hưởng mạnh nhất: tăng 1 SD $\rightarrow$ giá tăng $\approx 57.5\%$ ($e^{0.4547} - 1$). Tiếp theo là dung lượng RAM (+18.9%/SD), xung nhịp (+12.9%/SD), bus width (+7.3%/SD). GPU Nvidia đắt hơn AMD $\approx 45.2\%$ ($e^{0.3735} - 1$) cùng thông số. GPU ra sau rẻ hơn $\approx 3.8\%$ /năm (hiệu ứng giảm giá theo thời gian).

Kiểm định giả định: VIF tất cả biến < 3.0 (không có đa cộng tuyến). Breusch-Pagan: $BP = 99.04$, $p < 2.2 \times 10^{-16}$ (phương sai chưa hoàn toàn đồng đều). Shapiro-Wilk: $W = 0.948$, $p = 4.056 \times 10^{-13}$ (gần chuẩn hơn OLS, vẫn vi phạm nhẹ).

Nhận xét tổng thể: Mô hình log-linear cải thiện đáng kể so với OLS: R^2 tăng từ 29.8% lên 78.5%, W -statistic Shapiro-Wilk tăng từ 0.239 lên 0.948 (gần chuẩn hơn rõ rệt), và tất cả biến đều có ý nghĩa. Phương sai sai số vẫn chưa hoàn toàn đồng đều — đây là hạn chế còn lại, có thể khắc phục bằng Weighted Least Squares hoặc Robust Standard Errors nếu cần.

5 Kết luận và Đề nghị

5.1 Đánh giá mục tiêu nghiên cứu

5.1.1 Câu hỏi nghiên cứu đã được giải quyết

Đề tài đặt ra hai câu hỏi nghiên cứu chính và cả hai đã được trả lời thỏa đáng bằng bằng chứng thống kê rõ ràng:

Câu hỏi 1 — Thông số kỹ thuật nào ảnh hưởng đến giá GPU?
Mô hình Log-Linear ($R^2 = 78.54\%$) xác nhận cả 6 thông số đều có ý nghĩa ($p < 0.001$). TDP ảnh hưởng mạnh nhất ($+57.5\%/SD$), tiếp theo là dung lượng RAM, xung nhịp, bus width. Đặc biệt, `memory_bus` trở nên có ý nghĩa sau biến đổi log (vốn không có ý nghĩa trong OLS).

Câu hỏi 2 — Chênh lệch giá NVIDIA vs AMD? Welch's ANOVA ($F = 18.39$, $p = 2.409 \times 10^{-5}$): chênh lệch trung bình **\$245.43**. Mô hình hồi quy cho thấy cùng thông số, NVIDIA đắt hơn AMD **45.2%** — sự chênh lệch do chiến lược định giá chứ không chỉ do cấu hình.

Kết quả **vượt kỳ vọng**: $R^2 = 78.54\%$ (mục tiêu $\geq 70\%$). Bước chuyển từ OLS ($R^2 = 29.8\%$) sang Log-Linear là cần thiết do dữ liệu giá GPU có phân phối lệch phải cực đoan (Skewness = 16.84).

5.2 Hạn chế và đề xuất cải tiến

1. Phương sai sai số chưa đồng đều ($BP = 99.04$, $p < 2.2 \times 10^{-16}$). Đề xuất: WLS hoặc Robust SE (`vcovHC()`).



2. **Phần dư chưa hoàn toàn chuẩn** ($W = 0.948$, $p = 4.056 \times 10^{-13}$). *Đề xuất*: Biến đổi Box-Cox; CLT với $n = 556$ vẫn đảm bảo suy diễn hợp lệ.
3. **Selection bias** do loại 83.7% quan sát thiếu giá (MNAR). *Đề xuất*: Bổ sung dữ liệu từ Amazon/Newegg hoặc Multiple Imputation.
4. **Không xét tương tác biến**. *Đề xuất*: Thêm interaction terms (`tdp:manufacturer`) và Ramsey RESET test.

5.3 Ý nghĩa thực tiễn

1. **Công cụ định giá**: Phương trình Log-Linear cho phép ước lượng giá hợp lý dựa trên thông số kỹ thuật — nếu giá thực vượt xa giá mô hình, GPU có thể đang bị overpriced.
2. **Định lượng “phí thương hiệu”**: NVIDIA đắt hơn AMD $\approx 45.2\%$ cùng thông số — cơ sở để người mua đánh giá mức premium.
3. **Ngân sách doanh nghiệp**: Mô hình giúp so sánh giá trị thực giữa các GPU cho hệ thống AI/render farm.
4. **Tín hiệu thị trường**: GPU ra sau rẻ hơn $\approx 3.8\%$ /năm ($\beta_{year} = -0.039$), phản ánh xu hướng Moore’s Law.

Tài liệu

- [1] Ilias Kkaf. (2017). *Computer Parts (CPUs and GPUs)*. Kaggle. <https://www.kaggle.com/datasets/iliassekkaf/computerparts>. Truy cập lần cuối: tháng 4/2026.
- [2] Montgomery, D. C., Peck, E. A., & Vining, G. G. (2012). *Introduction to Linear Regression Analysis* (5th ed.). John Wiley & Sons.



- [3] R Core Team. (2024). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria.
<https://www.R-project.org/>.
- [4] Kutner, M. H., Nachtsheim, C. J., Neter, J., & Li, W. (2004). *Applied Linear Statistical Models* (5th ed.). McGraw-Hill/Irwin.
- [5] Field, A. (2018). *Discovering Statistics Using IBM SPSS Statistics* (5th ed.). SAGE Publications.